

# Module 07: Optimization & Advanced Ensembling - Part 3

Cybernauts 2026 Datathon Campaign

May 2026

## Abstract

Many machine learning practitioners mistakenly treat classification metrics as abstract mathematical values completely isolated from operational decision boundaries. Relying on default framework thresholds is one of the most common pitfalls in competitive data science. This note explores the mechanics of custom evaluation metrics, details the mathematical instability of the standard 0.5 decision boundary under class imbalances, and provides a template for constructing threshold-scanning optimization systems.

## Contents

|          |                                               |          |
|----------|-----------------------------------------------|----------|
| <b>1</b> | <b>The Arbitrary Default Threshold Trap</b>   | <b>2</b> |
| 1.1      | The 0.5 Sub-Optimality Flaw . . . . .         | 2        |
| <b>2</b> | <b>The Mathematics of Metric Optimization</b> | <b>2</b> |
| 2.1      | Deconstructing the $F_1$ -Score . . . . .     | 2        |
| <b>3</b> | <b>The Threshold Calibration Protocol</b>     | <b>2</b> |
| 3.1      | The Search Loop Strategy . . . . .            | 2        |
| <b>4</b> | <b>Practice Problems</b>                      | <b>4</b> |

# 1 The Arbitrary Default Threshold Trap

When evaluating a binary classification problem, tree-based ensemble models do not natively output discrete classes (0 or 1). Instead, they compute continuous probability distributions mapping instances to a risk scale spanning the interval  $[0, 1]$ .

## 1.1 The 0.5 Sub-Optimality Flaw

Standard machine learning API methods—such as Scikit-Learn’s default `predict()` function—automatically apply a rigid threshold cutoff at exactly 0.5. Any sample with a calculated probability greater than or equal to 0.5 is classified as a positive instance (1), while anything below is marked as a negative instance (0).

While a 0.5 boundary is intuitive for perfectly balanced distributions, it is completely sub-optimal for advanced competitive metrics like the  $F_1$ -Score, Macro F1, or Matthews Correlation Coefficient ( $MCC$ ), particularly when working with skewed or class-imbalanced datasets.

# 2 The Mathematics of Metric Optimization

To optimize a model effectively, we must evaluate how threshold shifts alter individual components of our classification metrics.

## 2.1 Deconstructing the $F_1$ -Score

The  $F_1$ -Score is defined as the harmonic mean of Precision and Recall:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN} \quad (1)$$

Where  $TP$ ,  $FP$ , and  $FN$  represent True Positives, False Positives, and False Negatives, respectively.

*The Optimization Dynamics:* Shifting your operational threshold creates a direct trade-off between these performance metrics:

- **Lowering the Threshold (e.g., to 0.35):** The model classifies more instances as positive. This aggressively drives down False Negatives ( $FN$ ) and increases Recall, but it runs the risk of inflating False Positives ( $FP$ ) and lowering Precision.
- **Raising the Threshold (e.g., to 0.65):** The model requires much higher confidence before assigning a positive label. This increases Precision by minimizing False Positives ( $FP$ ), but it typically causes Recall to drop due to an increase in False Negatives ( $FN$ ).

Because the  $F_1$ -Score treats Precision and Recall with equal weight, the optimal decision boundary is almost never located at an arbitrary 0.5 mark. It depends heavily on the specific density and distribution of your model’s prediction probabilities.

# 3 The Threshold Calibration Protocol

Instead of blindly accepting default values, competitive pipeline architectures use a post-processing optimization loop to systematically calibrate decision boundaries.

## 3.1 The Search Loop Strategy

The calibration workflow uses a post-processing search loop:

1. Extract the continuous out-of-fold validation probabilities from your ensemble using the `.predict_proba()[:, 1]` method.
2. Set up a search loop that scans across potential threshold parameters from 0.01 to 0.99 using a granular step size (such as 0.01).
3. At every step interval, convert the continuous probability vector into hard binary assignments (0 or 1) using the temporary threshold cutoff.

4. Calculate the local cross-validation score against the true validation targets at each interval.
5. Locate the precise coordinate that yields the absolute highest metric value, and lock in that threshold for your final submission file predictions.

#### The Validation Rule

You must calculate your optimal threshold coordinate using **Out-of-Fold (OOF) validation probabilities**. Never calibrate your threshold using training set probabilities, as this will lead to severe over-optimization and leaderboard performance drops.

If an optimized threshold of 0.38 yields a higher local validation  $F_1$ -score than 0.5, you discard the 0.5 default and use 0.38 as your hard operational boundary. This simple post-processing adjustment can drastically improve your leaderboard position in the final hours of a competition.

## 4 Practice Problems

**P1:** Write a clean, functional Python code block that implements a threshold scanning optimization loop. The function must accept an out-of-fold probability vector (`oof_preds`) and the true target labels (`y_true`), evaluate thresholds from 0.01 to 0.99 at 0.01 intervals, and return the absolute best threshold alongside its corresponding maximum  $F_1$ -score.

*Answer:*

```
import numpy as np
from sklearn.metrics import f1_score

def optimize_threshold(y_true, oof_preds):
    best_threshold = 0.5
    best_score = 0.0

    # Grid scan candidate thresholds from 0.01 to 0.99
    for threshold in np.arange(0.01, 1.00, 0.01):
        # Convert continuous probabilities into binary choices
        preds_binary = (oof_preds >= threshold).astype(int)

        # Evaluate performance metric
        score = f1_score(y_true, preds_binary)

        # Track maximum performance coordinate
        if score > best_score:
            best_score = score
            best_threshold = threshold

    return best_threshold, best_score
```

**P2:** A model evaluates a rare disease classification dataset. The out-of-fold validation array contains a small number of positive cases. A baseline evaluation at a default threshold of 0.5 yields  $TP = 10$ ,  $FP = 2$ ,  $FN = 15$ . Calculate the current  $F_1$ -score, and predict what happens to the score if lowering the threshold to 0.30 changes the values to  $TP = 22$ ,  $FP = 8$ ,  $FN = 3$ .

*Answer: First, calculate the baseline  $F_1$ -score at the 0.5 threshold:*

$$F_{1\_base} = \frac{2 \times 10}{(2 \times 10) + 2 + 15} = \frac{20}{37} \approx 0.5405$$

*Next, calculate the calibrated  $F_1$ -score at the adjusted 0.30 threshold:*

$$F_{1\_calibrated} = \frac{2 \times 22}{(2 \times 22) + 8 + 3} = \frac{44}{55} = 0.8000$$

*Lowering the threshold to 0.30 successfully captures missing positive cases, causing the  $F_1$ -score to jump from 0.5405 to 0.8000. This demonstrates why optimizing your decision boundary is so critical.*

**P3:** During an optimization phase, an engineer calculates the optimal decision threshold using the model's training set predictions, finding a peak  $F_1$ -score at a threshold of 0.24. When evaluated on out-of-fold validation splits using this 0.24 threshold, the model's performance drops drastically. Explain the statistical flaw behind this drop.

*Answer: The engineer optimized the threshold parameter against the training set distribution rather than the out-of-fold validation distribution. Because models naturally lean toward overconfidence on their training data, training set probabilities are heavily overfitted. Calibrating your decision boundary on this biased data picks a threshold that is optimized for training noise, creating an unstable boundary that fails to generalize to unseen validation or test sets.*